Assignment A4: RL-ing Nanochat

Markson Chen (1009002598)
Kevin Liu (1008495237)
Barry Jiang (1008774097)
Yibin Cui (1008826512)

Part One

At the start of `scripts/chat_rl.py`, Karpathy outlined the main changes he made compared to the Standard GRPO:

1. *Delete trust region, so there is no KL regularization to a reference model.*
2. *We are on policy, so there's no need for PPO ratio+clip.*
3. *We use DAPO style normalization that is token-level, not sequence-level.*
4. *Instead of z-score normalization (r - mu)/sigma, only use (r - mu) as the advantage.*

Instead of rephrasing them here, we will discuss our own understandings of these four changes:

1. The KL loss term pulls the model close to the original SFT model. Having it can preserve other functionalities trained during SFT, like the general conversational abilities provided by the SmolTalk dataset. Removing it just means we don't want to preserve generalizability outside the GSM8K task.
2. The PPO ratio has dropped because the ratio $r_t(\theta) = \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t})}$ comes from importance sampling, where we estimate the new policy using the old policy's distribution. Since in Karpathy's script, we update on the same policy we sample from, $r_t(\theta_{\text{old}}) = 1$ and the ratio can be dropped.
3. Computing the loss by averaging across tokens instead of averaging across groups first makes longer responses contribute more toward the gradient. If it is empirically better, perhaps one reason could be that we are assigning more credits to a longer chain of reasoning.
4. Given that the current RL script is only using binary rewards, removing rescaling makes sense. For example, if all samples are wrong or correct, then $\sigma_q = 0$. So z-scoring is an unstable gradient scaling term that can be noisy.

Part Two

We choose to train nanochat on **d18** rather than larger models, because at d18, we can already see meaningful differences in GSM8K accuracy. We used the version of on the code at March 4.

The instructions for this part are a bit open-ended, and we decided to focus only on improving the **GSM8K task**.

First, running SFT on d18 with the original configuration resulted in an accuracy of **3.79%**.

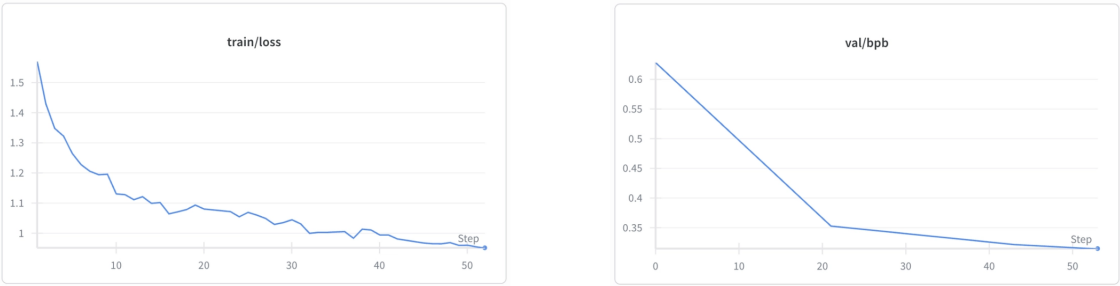

Figure 1. The Training and Validation Loss Curves of a Standard d18 SFT Run

To improve nanochat on GSM8K, we chose **Orca-Math** (Mitra et al., 2024) because the paper reports 79.91% accuracy on GSM8K after just one SFT epoch. To maintain a controlled ablation study, we replaced the SimpleSpelling and SpellingBee datasets with Orca-Math by **keeping the number of tokens the same**. (We found 200K rows of SimpleSpelling + 80K rows of SpellingBee of the original data mixture matches with 61K rows of Orca-Math on the token level.)

However, the initial run of this new data mixture resulted in a GSM8K accuracy of just **0.38%**. After querying the model, we realized that the ground-truth responses of Orca-Math do not have the **"#### answer" format** that GSM8K requires. So we wrote a script to extract the final numerical answer and correctly format it, and rerun the training.

The new run resulted in an accuracy of **3.18%**, lower than the original data mixture (3.79%). We hypothesize that the reason could be: GSM8K emphasized on producing step-by-step reasonings with <<expr=result>> structures, which mismatched with Orca-Math. Learning two paradigms of chain of reasoning without conflicting may require a larger model.

The loss/eval curves for the three data mixtures are shown below (logged using Weights and Biases). They are having a roughly similar loss curve and validation curve, which makes sense given that it is an ablation experiment with all the hyperparameters and the total number of training tokens controlled.

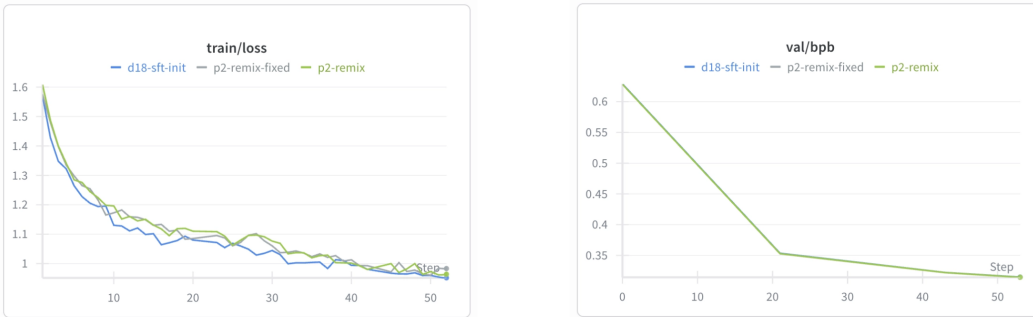

Figure 2. The Training and Validation Loss Curves of Three d18 SFT Runs (**p2-init**: The original data mixture; **p2-remix**: Replacing SimpleSpelling + SpellingBee with Orca-Math; **p2-remix-fixed**: Fixed the answer formatting of Orca-Math)

Part Three

Running chat_rl.py with the original configurations on the SFT d18 model trained on the original data mixture resulted in 11.37% accuracy on GSM8k.

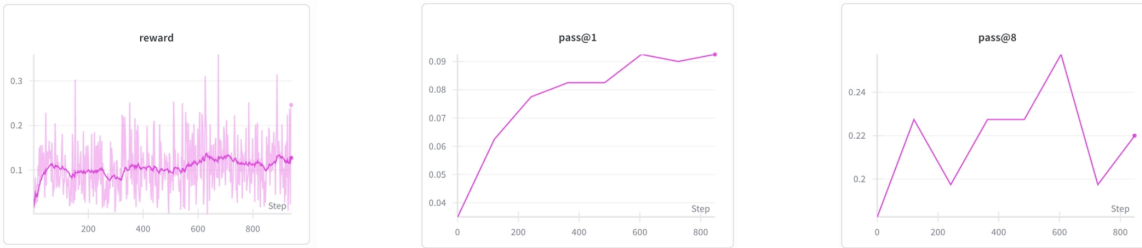

Figure 3. The Reward, Pass@1, Pass@8 Curves of a Standard d18 RL Run

The results are better than Karpathy's original run on Oct 13, 2025, which got **7.58%** accuracy in GSM8k from a d20 model. This is expected from the series of improvements Karpathy made over the 5 months.

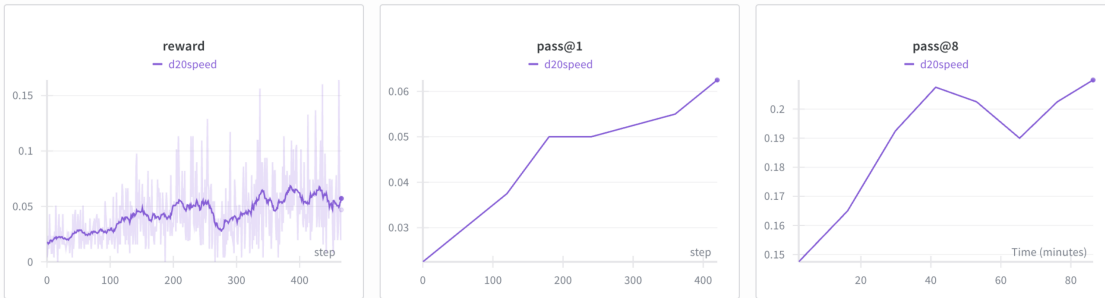

Figure 4. The Reward, Pass@1, Pass@8 Curves of a d20 RL Run On Oct 13 (Shared by Karpathy)

The errors can largely be grouped into two classes: Unstructured responses and math errors.

1. Unstructured responses: The model produces unstructured rambling prose and fails to arrive at a `#### N` answer format. For example:

> *Q: An ice cream truck is traveling through a neighborhood. Children from various homes have started chasing the truck and, by the time the truck has reached the end of the first street, there are 5 children it. On the second street, each child is joined by another child and on the third street, each child in the existing group is joined by another 2 children. The original 5 children then give up and leave the group. How many children are now following the truck?*
>
> *Model response: To find the number of children following the truck, we need to consider the initial 5 children and the subsequent children.*
>
> *Initially, there are 5 children. When the truck reaches the end of the first street, each child is joined by another child, so there are 5 children in the first street.*
>
> *When the truck reaches the end of the second street, each child is joined by another child, so there are 5 children in the second street.*
>
> *When the truck reaches the end of the third street, each child is joined by another child, so there are 5 children in the third street.*
>
> *When the truck reaches the end of the fourth street, each child is joined by another child, so there are 5 children in the fourth street.*
>
> *When the truck reaches the end of the fifth street, each child is joined by another child, so there are 5 children in the fifth street.*
>
> *Now, we need to find the number of children following the truck. Since there are 5 children in the first street, 5 children in the second street, and 5 children in the third street, we can calculate the number of children following the truck as follows:*
>
> *5 children in the first street + 5 children in the second street + 5 children in the third street + 5 children in the fourth street = 15 children.*

2. Math error: The model correctly outputs steps with structured calculator usages and answer formats, but writes the wrong math. If we were to further classify the math errors, the types of errors include: misordered operations, number copying errors, and skipped steps. An example is shown below (Computes Shaelyn as $14 - 2$ instead of $14 / 2$).

> *Q: Bethany can run 10 laps on the track in one hour. Trey can run 4 more laps than Bethany. Shaelyn can run half as many laps as Trey. Quinn can run 2 fewer laps than Shaelyn. How many more laps can Bethany run compared to Quinn?*
>
> *Model response: Trey: 10 + 4 =*
> Trey: 10 + 4 = <|python_start|>10+4<|python_end|><|output_start|>14<|output_end|>14 laps
> Shaelyn: 14 - 2 = <|python_start|>14-2<|python_end|><|output_start|>12<|output_end|>12 laps
> Quinn: 12 / 2 = <|python_start|>12/2<|python_end|><|output_start|>6.0<|output_end|>6.0 laps
> Quinn: 6 - 2 = <|python_start|>6-2<|python_end|><|output_start|>4<|output_end|>4 laps
> Quinn: 4 - 14 = <|python_start|>4-14<|python_end|><|output_start|>-10<|output_end|>-10 laps
> Quinn: 10 - 6 = <|python_start|>10-6<|python_end|><|output_start|>4<|output_end|>4 more laps
> #### 4

Part Four

The original chat_rl.py only uses a binary correctness reward. We added two more rewards:

1. **Format Reward $\mathbf{R_f}$**: 1.0 if there is exactly one #### answer and it is on the final line; else 0.0. This reward is supported by our observation in Parts 2 and 3, in that many outputs failed to follow the answer format expected by GSM8K.
2. **Self-Consistency Reward $\mathbf{R_{sc}}$**: $R_{sc,i} = \frac{1}{K}\sum_{j=1}^{K} \mathbf{1}[\hat{a}_j = \hat{a}_i]$. The paper that proposes this reward (Wang et al., 2022) shows that it helps with chain of thought reasoning, and leads to a +17.9% increase in GSM8K.

Let $\boldsymbol{R_c}$ denote the existing reward of binary correctness. We run four runs:

- **d18-rl-init**: $R = R_c$
- **d18-rl-A**: $R = R_c + 0.02R_f$
- **d18-rl-B**: $R = R_c + 0.05R_{sc}$
- **d18-rl-AB**: $R = R_c + 0.02R_f + 0.05R_{sc}$

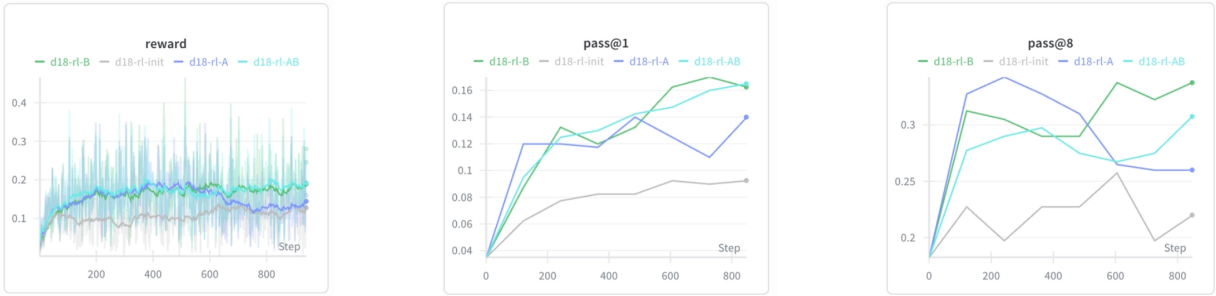

Figure 5. The Reward, Pass@1, Pass@8 Curves of The Four d18 RL Runs

	GSM8K Accuracy	pass@1	pass@8
d18-rl-init	11.37%	0.0925	0.22
d18-rl-A	16.22%	0.14	0.26
d18-rl-B	**16.68%**	0.1625	0.3075
d18-rl-AB	16.07%	**0.165**	**0.3375**

Table 1. The GSM8K Accuracy, Pass@1, Pass@8 results of The Four d18 RL Runs (The pass@K results are evaluated on a subset of 400 questions in the test set)

Observing the model outputs, we notice that, introducing the format reward eradicated all formatting errors. In the SFT run, 4 our of 10 outputs lack the correct tool use and answer formatting, but in the outputs of d18-rl-A and d18-rl-AB, interestingly, all outputs have the correct <|python_start|><|python_end|><|output_start|><|output_end|> tool use formats and the #### answer formats. However, all sorts of math errors still appear.

RL Model Trained	What we've noticed
d18-rl-init	Formatting errors, unstructured responses, math errors
d18-rl-A	No formatting errors, still all sorts of math error
d18-rl-B	(Less errors) Formatting errors, unstructured responses, math errors
d18-rl-AB	No formatting errors, still all sorts of math error